OPIMQ: Order Preserving IO stack for Multi-Queue Block Device

Jieun Kim, *Korea Advanced Institute of Science and Technology* (*KAIST*); Joontaek Oh, *University of Wisconsin–Madison;* Juwon Kim, Seung Won Yoo, and Youjip Won, *Korea Advanced Institute of Science and Technology* (*KAIST*)

https://www.usenix.org/conference/fast25/presentation/kim-jieun

This paper is included in the Proceedings of the 23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings of the 23rd USENIX Conference on File and Storage Technologies is sponsored by

OPIMQ: Order Preserving IO stack for Multi-Queue Block Device

Jieun Kim[1], Joontaek Oh*[2], Juwon Kim[1], Seung Won Yoo[1], and Youjip Won[1]

[1]Korea Advanced Institute of Science and Technology
[2]University of Wisconsin-Madison

Abstract

In this work, we address the issue of ensuring the storage order in the multi-queue IO stack and propose *OPIMQ*, an Order-Preserving IO Stack for Multi-Queue Block Devices. OPIMQ consists of four key components: *Epoch Pinning*, *Dual-Stream Write*, *Order-Preserving Mapping Table Update* and *Sibling-aware Delayed Mapping*. With Epoch Pinning, we can preserve intra-stream order dependency across different queues. With Dual-Stream Write, we can preserve the inter-stream order dependency across different threads. With Order-Preserving Mapping Table Update, FTL can update the mapping table with respect to the storage order. With Sibling-Aware Delayed Mapping, FTL can update the mapping table only when the dual-stream write satisfies the storage order in both streams. Linux IO stack with OPIMQ outperforms the vanilla Linux IO stack by 2.9×, 2.8×, and 2.9× under Filebench `varmail`, `dbench`, and `sysbench`, respectively. The order-preserving FTL accompanies a 1.1% performance penalty in address translation compared to legacy FTL.

1 Introduction

We observed phenomenal advancements in storage device technology from the perspective of both performance and capacity. For the past several decades, the storage performance has increased by 1800× (from 300 IOPS HDD to 550K IOPS Intel Optane 900P [4]). It is thanks to the concurrency and parallelism introduced in storage devices: multiple command queues [5, 6], as well as the wider channel/way parallelism of storage controllers [14].

In many applications, ensuring the order in which a set of blocks becomes durable, *storage order*, is an essential requirement, e.g., filesystem journaling [33], log-structured filesystem [30], soft-update [26], and database transaction [28]. The application ensures the storage order through a very costly approach: interleaving the write requests with *Transfer-and-Flush* [34]. It dispatches the following request only after the data blocks associated with the preceding request have been transferred to the storage device and made durable. However, this approach neutralizes various concurrency and parallelism features of the modern storage device. Enforcing the storage order among write requests prevents the application from fully leveraging the performance capabilities of the underlying storage.

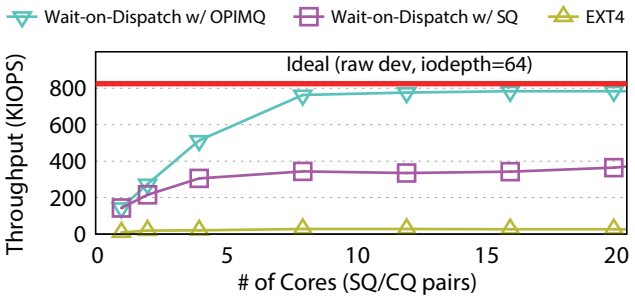


Figure 1: 4KB random write throughput with storage order guarantee: EXT4 with Transfer-and-Flush (EXT4), EXT4 with Wait-on-Dispatch in Single Command Queue (Wait-on-Dispatch w/ SQ), and EXT4 with Wait-on-Dispatch in OPIMQ (Wait-on-Dispatch w/ OPIMQ)

A few works propose the storage order guarantee mechanisms that dispense with Transfer-and-Flush [11, 24, 25, 34]. BarrierFS [34] exploits the `cache barrier` command to ensure the storage order and eliminates the need for Transfer-and-Flush to satisfy the storage order constraint. However, BarrierFS works only in a single-queue IO stack, e.g., eMMC [8] and SATA [7].

The modern IO stack adopts a multi-queue organization, where the host defines the multiple request queues [5], and the storage device defines multiple command queues [6]. Single-queue-based order-preserving mechanisms leave substantial room for improvement in leveraging the inherent parallelism and scalability of multi-queue IO stacks. We examine the performance of three different storage order guarantee mechanisms: Transfer-and-Flush, Wait-On-Dispatch with a single queue, and Wait-on-Dispatch with OPIMQ. To examine the many-core scalability of the storage order guarantee mechanism, we vary both the number of threads and the number of command queues in the storage device from one to forty. Figure 1 illustrates the results. *Ideal* represents the case where the system achieves the maximum throughput: there are forty threads, and each thread dispatches the next write immediately after dispatching the previous one. Legacy EXT4 ensures the storage order using Transfer-and-Flush. In Wait-On-Dispatch,

*This work was done while the author was a graduate student at KAIST.

the benchmark dispatches the next request immediately after dispatching the previous write. In Wait-On-Dispatch with a single queue, all requests are directed to the same command queue. The overhead of Transfer-and-Flush is severe. When Transfer-and-Flush is used to ensure the storage order, random write throughput renders only 3.4% of the ideal throughput. The performance of Wait-On-Dispatch with a single queue reaches only 47% of that of the ideal throughput. These limitations highlight the challenges of ensuring storage order in modern multi-core, multi-queue environments.

Ensuring storage order in multi-queue block devices presents significant challenges: the IO stack must guarantee storage order not only within individual queues but also across multiple queues. This inter-queue storage order is particularly challenging because each queue operates independently and is scheduled without coordination with others. HORAE [24] and MQFS [25] address the problem of ensuring storage order in the multi-queue IO stack. Their approaches either unnecessarily impose storage order on irrelevant requests [24] or may lead to severe load imbalances across request queues and CPU cores [25].

In this work, we develop an order-preserving IO stack for the multi-queue block device, *OPIMQ*. OPIMQ is grounded upon the *cache barrier* command to ensure the storage order. The essential problem is to ensure the storage order between the write requests that are placed at different queues. The write requests at different queues can be either from the same thread or from the different threads. The former and the latter are called *Intra-Stream Store Order Guarantee* and *Inter-Stream Storage Order Guarantee*. For intra-stream storage order guarantee, we develop *Epoch Pinning*. Epoch Pinning places IO requests in the same epoch at the same request queue even though the thread is migrated to another core. For inter-stream storage order guarantee, we develop *Dual-Stream Write*. Dual-stream write is a special type of write request that belongs to two streams simultaneously. Order-preserving FTL, *OPFTL*, is responsible for guaranteeing intra-stream order and inter-stream order of the IO commands. With *Order-preserving Mapping Table Update*, OPFTL guarantees that the data blocks become durable in order. With *Sibling-Aware Delayed Mapping*, OPFTL ensures that the dual stream write is made durable in order to satisfy the ordering constraints of the two streams that it is associated with.

Combined all these together, OPIMQ can ensure the storage order in the multi-queue IO stack fully exploiting the performance potential of the underlying storage device. In ensuring the storage order, OPIMQ allows the OS to freely migrate a thread from one core to another and allows the IO requests that are dependent on each other to be placed at different request queues. The contribution of our work can be summarized as follows:

- **`cache barrier` command for Multi-Queue SSD.** OPIMQ opens up an new avenue for the `cache barrier` command to be employed in the storage products for the server and the data center environment. Despite its significant potential benefit, `cache barrier` command has been employed in only the mobile storage products since it has been effective only in the single-queue IO stack.
- **Exploiting the internal parallelism of SSD.** OPIMQ ensures both intra-stream and inter-stream order dependencies while maximizing the utilization of multiple queues and CPUs. OPIMQ enables the host to fully exploit the rich internal bandwidth of modern Flash storage.
- **Mitigating the overhead for storage order guarantee** OPIMQ replaces costly Transfer-and-Flush mechanism with a lighter weight `cache barrier` command. It improves the performance and scalability of the cloud applications.

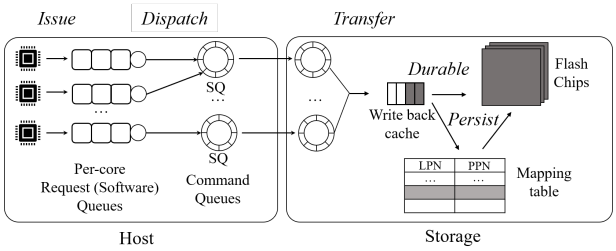


Figure 2: Multi-Queue IO Stack: A concept, SQ: submission queue

We implement OPIMQ on Linux 5.18.18. EXT4 filesystem ported over OPIMQ achieves up to 2.8× and 2.9× higher throughput than EXT4 in `dbench` and `sysbench`, respectively.

2 Background and Related Works

2.1 Multi-Queue IO Stack

Multi-Queue Block Device Layer. Since Linux 4.16, the Linux kernel has defined more than one request queue at the block device layer [5]. Each request queue is scheduled independently. This is to mitigate the lock contention on the request queue among the multiple cores. In the current implementation, the Linux OS creates a request queue for each CPU core and statically binds a request queue to each core. IO requests are inserted to the request queue attached to the CPU core where the associated thread is running.

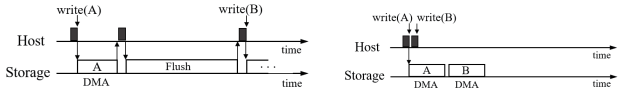
Multi-Queue Block Device. Modern storage devices adopt parallel internal organization, including multi-channel/way architecture [17, 20, 23], and large writeback caches. To fully utilize the rich internal bandwidth of the storage device, the recent storage interfaces, e.g., NVM Express (NVMe) [6], and UFS [16], export multiple command queues to the host. The NVMe storage device can export up to 64K SQ/CQ pairs, and each queue can have up to 64K commands [6]. The storage controller services the commands from the queues independently. Typical scheduling policy includes Round-Robin [6], Weighted Round-Robin [6, 18, 35], etc.

Multi-Queue IO Stack. Fig. 2 illustrates the organization of the Multi-Queue IO stack. The IO stack consists of four layers: filesystem, block device layer, writeback cache (device side), and storage media. An application invokes a system call to request an IO. Then, the filesystem constructs the IO request (e.g., `struct bio` in Linux) for the system calls from the application (*Issue*). Based on the IO request (`struct bio`) from the filesystem, the block device layer constructs a request object (e.g., `struct request`) and places it in the per-core request queue. The device driver associated with the storage device removes the request from the request queue, constructs the command, e.g., `struct nvme_command`, and places it in the command queue (*Dispatch*). The storage controller removes the command from the command queue and services it (*Transfer)*, e.g., transferring the data from the host to the writeback cache in the storage device (write) or transferring the data from the storage device to the host (read). The storage controller flushes the data blocks from the writeback cache (*Durable*). This occurs either through its own internal periodic activity or by an explicit request from the host, e.g., `flush` command. Finally, the storage controller updates the mapping table so that the host can access the flushed data blocks (*Persist*).

2.2 Storage Order Revisited

The storage device exports a well-defined, rigid interface [6, 7, 16, 29] to the host computer. This strict layered organization separates the storage device from the host computer and allows the storage system to evolve independently of the host. Although the independent isolation between the storage system and the host computer has led to phenomenal advancements in storage device technology, it also creates significant challenges for collaboration between the host computer and the storage device.

Ensuring the storage order is an end-to-end requirement. To ensure the storage order imposed by the application, each layer of the IO stack, the filesystem, the block layer, and the storage device, needs to satisfy a certain partial order in a conjunctive manner. However, the strict layered organization of the modern IO stack makes it practically impossible to pass the ordering constraint from one layer to another and to enforce the other layers to preserve the partial order that one layer has passed. Due to this reason, the applications need to enforce the storage order using a highly expensive method: interleaving write requests with the costly Transfer-and-Flush mechanism (Fig. 3a). It neutralizes various concurrency and parallelism features of the modern storage device, e.g., deep command queue, large writeback cache, multi-channel/way controller. When the application needs to ensure the storage order among the write requests, it fails to fully utilize the performance potential of the underlying storage. When the IO stack can preserve the partial order across the layers, the application can ensure the storage order without Transfer-and-Flush overhead (Fig. 3b).



(a) Wait for Transfer-and-Flush (b) Wait for Dispatch

Figure 3: Guaranteeing the Storage Order

3 Problem Formulation

In this work, we develop a mechanism that ensures the storage order in the multi-queue block device. We like to ensure the storage order without relying on the transfer-and-flush. The key challenge is preserving the storage order among the requests placed at the different command queues. Assume that the write requests, which must adhere to a specific storage order, are inserted into different command queues. The transfer order of the write requests can be rearranged by the storage controller, according to its scheduling policy, potentially deviating from their issued order. The storage order dependency across the different command queues can arise in two cases: intra-stream storage order and inter-stream storage order.

We use barrier-EXT4 to demonstrate the issues and challenges with guaranteeing intra-stream storage order and inter-stream storage order. Barrier-EXT4 is cache-barrier-enabled version of EXT4. In committing a journal transaction, EXT4 writes the dirty pages, journal log blocks, and a journal commit block in order. To ensure the storage order between the dirty pages and the journal log blocks, JBD thread starts writing the journal log blocks only after all write requests for the dirty pages are serviced. To ensure the storage order between the log blocks and the commit block, JBD thread writes the journal commit block after the journal log blocks become durable. In barrier-EXT4, both of these order-preserving mechanisms are replaced with `cache barrier`. In barrier-EXT4, the filesystem issues `cache barrier` command right after it issues the write request for dirty pages. Then, the JBD thread starts writing the log blocks without waiting for the write requests for the dirty pages to finish. Likewise, the JBD thread issues a cache barrier command after it issues the write requests for log blocks. Then, it writes the journal commit block without waiting for the journal log blocks to become durable. Barrier-EXT4 works correctly in the single queue order-preserving IO stack [34].

3.1 Model

A *stream* is a set of IO requests from the same thread. We define write requests that must be subject to ordering constraints as *ordered writes*. We also call the write requests that are not associated with any ordering constraints *unordered writes*. There are four cases where the page cache contents are written to the disk: periodic page cache flush (`pdflush`), periodic journaling (`JBD`), `sync()` call and `fsync()` call. The `pdflush`

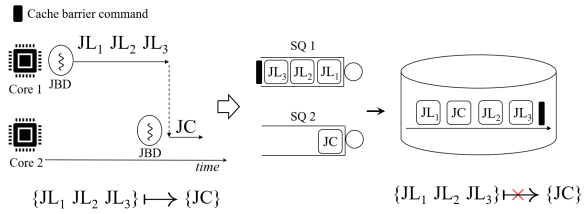


Figure 4: Stream bounce and intra-stream storage order; JBD thread in barrier-EXT4

daemon and `sync()` create the unordered write requests. An `fsync()` or periodic journal commit create the ordered write requests.

An *epoch* is a set of ordered write requests that can be reordered or coalesced with each other. The cache barrier command delimits the epoch for each stream. Write requests enclosed within "{}" correspond to a single epoch. E_k represent the k^{th} epoch. To represent the order of two epochs, we use the "$\mapsto$" notation. All write requests that belong to the preceding epoch must be persisted in storage before any write requests from the subsequent epoch.

3.2 Intra-Stream Storage Order Guarantee

The operating system uses the work stealing [10] to balance the load among the multiple cores in the system. The write requests in a stream can be placed in different queues due to the work stealing, and a stream can be split over multiple queues. We refer to this phenomenon as *stream bounce*.

We explain how the stream bounce can compromise the storage order among the requests in a stream. Fig. 4 illustrates an example. For ease of explanation, we use the barrier-EXT4. Suppose that the JBD commits the running transaction and creates three write requests for the journal log blocks (JL_1, JL_2, JL_3), and one write request for the journal commit block (JC). The journal log blocks and the journal commit block should be made durable in order, i.e., $\{JL_1, JL_2, JL_3\} \mapsto \{JC\}$. Assume that the work-stealing module schedules the JBD thread from core 1 to core 2 after the JBD inserts the write requests for JL_1, JL_2, JL_3, and a cache barrier command. These requests are placed at SQ1. After the JBD thread is migrated to core 2, it generates the write

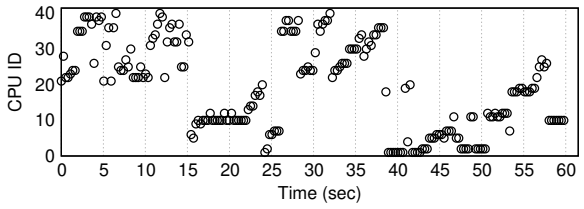


Figure 5: Working stealing in reality: CPU ID where the JBD is running, `varmail` workload with 20 threads, 40 core server

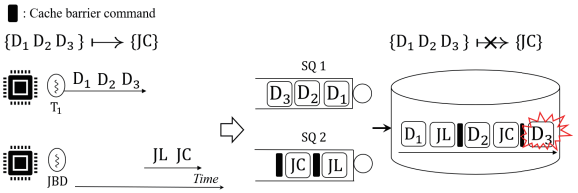


Figure 6: Inter-stream order dependency between `fsync()` caller thread T_1 and the JBD, in barrier-EXT4

request for the journal commit block (JC). The write request for the journal commit block is placed at SQ2. If the storage controller services the requests in SQ1 and SQ2 in a round-robin fashion, the write request for the journal commit block, JC, is serviced before the JL_2 and JL_3 are. Subsequently, the journal commit block can become durable while some of the log blocks are in transit. The storage order of $\{JL_1, JL_2, JL_3\} \mapsto \{JC\}$ can be compromised.

The stream bounce is the norm rather than an exception. We run `varmail` [32] benchmark with twenty threads. The server has 40 CPU cores. `varmail` benchmark is known for its frequent `fsync()` calls. We set the number of application threads much smaller than the number of CPUs in the system. This is to prohibit the work-stealing module from migrating the threads unnecessarily. We keep track of the CPU core on which the JBD thread runs. Fig. 5 illustrates the results. Contrary to our expectations, the work-stealing module keeps migrating the JBD thread from one core to another even though the total number of threads in the system is much smaller than the number of cores in the system. Detailed analysis of the working-stealing algorithm of Linux OS is beyond the scope of this work. In this experiment, we find that the JBD thread switches the CPU core approximately once every 120 ms (532 times in 60 seconds).

3.3 Inter-Stream Storage Order Guarantee

The write requests from different threads may bear ordering dependency among them. We call this situation an inter-stream storage order.

We explain how the inter-stream storage order can be compromised in a multi-queue IO stack. We use barrier-EXT4 journaling as an example. In EXT4 (and barrier-EXT4) journaling, dirty pages should be made durable ahead of the journal commit block. There is an ordering dependency between the two different streams: the write requests from the application thread and the write requests from the JBD thread. Fig. 6 illustrates an example. An application thread, T_1, calls `fsync()`. It issues the write requests for the dirty pages, D_1, D_2, and D_3. Immediately after dispatching the write request D_3, T_1 wakes up the JBD thread. JBD thread is deployed at a different core than T_1. The JBD thread dispatches one write request for the log block (JL). Then, it dispatches the write request for the commit block (JC). There exists the storage

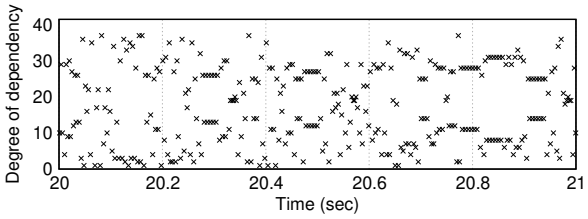


Figure 7: Degree of dependency in inter-stream storage order, `varmail` workload with 40 threads, 40 core server, in barrier-EXT4

order dependency between the write requests from T_1 and the write request from JBD, $\{D_1, D_2, D_3\} \mapsto \{JC\}$. If the storage controller services the requests at the queues in a round-robin fashion, D_3 may be serviced after the *JC* is serviced. Then, the journal commit block, *JC* can be made durable before all dirty pages become durable. The storage ordering constraint can be compromised.

In inter-stream storage order, a following stream can have more than one preceding stream; a write request from a stream needs to be made durable after a set of write requests from multiple streams all become durable. In the case of barrier-EXT4, the application thread T_1 and the JBD thread correspond to a preceding stream and a following stream, respectively. We call this situation *many-to-one inter-stream order dependency*. An example of such a scenario can be observed in the compound transaction of the EXT4. In EXT4, all metadata changes for every file in the filesystem are coalesced into a single global transaction. The commit of a single transaction involves the multiple `fsync()` operations for each file. In this case, a single write request for the journal commit block from the JBD should be durable after the write requests for the dirty pages from multiple caller threads.

We examine the degree of dependency of inter-stream storage order in barrier-EXT4. We run the `varmail` workload with forty threads. In this experiment, we collect the number of threads that are associated with the journal transaction for each transaction commit. Fig. 7 illustrates the results. We observe that the degree of dependency varies widely, with the average degree of dependency being seventeen.

3.4 Limitations of Existing Work

Several works have proposed a mechanism to reduce the overhead of guaranteeing the storage order [12, 24, 25, 34, 36]. We compare the existing designs for order-preserving IO stack. Table 1 summarizes the results.

	BarrierIO [34]	HORAE [24]	RIO [36]	ccNVMe [25]	OPIMQ
Multi-queue device	x	o	o	o	o
Multi-streams	x	x	x	o	o
Intra-stream depend.	x	o	o	o	o
Inter-stream depend.	x	x	x	x	o
Operate without PMR	o	x	x	x	o

Table 1: Comparison of Different Order-Preserving Techniques. PMR: persistent memory region of the NVMe SSD.

BarrierFS [34] is the seminal work on order-preserving IO stack. BarrierFS models a storage order as a conjunction of issue order, dispatch order, transfer order, and persist order. They propose a set of mechanisms for satisfying the constraint associated with each of these ordering constraints. It uses the cache barrier command to ensure the storage order. BarrierIO stack is designed for a single command queue block device and cannot be used in the multi-queue IO stack.

OPTR [11] assigns a unique id to the write requests, imposing the global order on all of them. The id is stored in the spare area of each flash page. When a system crashes, the recovery module recovers the filesystem state with respect to the id in the spare area, ensuring the storage order. It associates all IO requests in the global single stream. This approach fails to fully exploit the parallelism of the multi-queue block device.

HORAE [24], RIO [36] and ccNVMe [25] separate the ordering constraints from the associated data blocks and log the ordering constraints at the Persistent Memory Buffer (PMR) of the NVMe device. To handle the storage order dependency among the streams, HORAE resorts to aggregating all write requests from the threads into a single global stream. HORAE may unnecessarily impose ordering constraints on unrelated write requests. RIO [36] extends HORAE to guarantee ordering between storage nodes. RIO shares the same limitations with HORAE in ensuring the storage order among the streams.

ccNVMe addresses the limitation of HORAE in that ccNVMe allows each thread to have its own stream. ccNVMe prevents the inter-stream storage order. The filesystem ported on ccNVMe, MQFS, commits a journal transaction in its own context instead of delegating the journal commit to the dedicated thread. In the absence of a dedicated journaling thread, it becomes impossible to commit multiple file operations with a single transaction, and each `fsync()` separately commits its own transaction. When a large number of threads call `fsync()` concurrently, the storage device is flooded with an excessive number of flush commands, and the journaling throughput fails to scale.

4 OPIMQ: Design and Implementation

4.1 Design Goals

We develop an Order-Preserving IO Stack for a Multi-queue storage device, OPIMQ. We establish three design objectives in OPIMQ. First, we aim to preserve the storage order without expensive Transfer-and-Flush. Second, OPIMQ should be able to fully exploit the multiple request queues at the block layer. Third, OPIMQ should be able to fully utilize the multiple command queues at the storage device.

Mitigating the overhead of ensuring the storage order. OPIMQ exploits the *cache barrier* command to ensure the storage order between the write requests. When the filesystem needs to ensure the storage order between writing the blocks, e.g., between writing the dirty pages and writing the journal

commit block in EXT4 journaling [2] or between writing the node blocks and writing the data blocks in F2FS [22], OPIMQ allows the filesystem to use the cache barrier command. In this work, we port EXT4 over OPIMQ to examine the effectiveness of OPIMQ in realizing the order-preserving IO stack. OPIMQ is filesystem agnostic. Any filesystem can be ported over OPIMQ.

Exploiting Multiple Request Queues at the block layer. OPIMQ allows the write requests to be placed in the different request queues even though there exists a dependency among them. OPIMQ allows the thread to be freely scheduled to the different cores. Existing works either place the requests with any dependency at the single request queue or pin the thread at the core to ensure the storage order. This approach fails to fully utilize the multiple request queues at the block layer, multiple CPU cores at the system, or both. OPIMQ minimizes imposing unnecessary ordering constraints on the irrelevant requests and imposes partial orders only among the relevant requests.

Exploiting multiple command queues at the storage device. In OPIMQ, the host fully exploits the multiple command queues of the storage device when sending the IO commands to the storage device. In OPIMQ, the storage device persists the data blocks in the writeback cache, fully exploiting its internal bandwidth, and yet can preserve the storage ordering dependency among them.

4.2 Design Overview

OPIMQ consists of a file system, a block device layer, a device driver, and FTL. OPIMQ defines a stream and an epoch as a set of write requests from the same thread and the set of write requests in a stream that can be reordered with each other.

Filesystem specifies two attributes for a write request: whether a given write request is subject to ordering constraints or not, and whether a given write request is the last request in an epoch. For this purpose, OPIMQ introduces two flags: the *ordered flag* and *barrrier flag*. *ordered flag* and *barrrier flag* denote whether a given write request is ordered write or not and whether a given write request is the last write in an epoch. If the write request is the last one in the epoch, the barrier flag is set. The write request with the barrier flag set is called barrier write.

The block layer in OPIMQ is responsible for specifying the ordering constraints on the write request. The block layer assigns <stream id, epoch id> for a write request. This pair of id is used to represent the ordering constraint among the write requests.

The device driver constructs the IO command based upon the `struct bio` object from the block layer. We allocate the 8-byte unused space in the NVNe command object for two <stream id, epoch id> pairs. We need the space for two pairs of <stream id, epoch id> because of dual stream write (section 4.6.1). Instead of implementing a cache barrier as a separate command, OPIMQ uses the barrier write command [34]. It allocates an unused bit in the NVMe command as a barrier flag. In barrier write, OPIMQ sets the barrier flag on the write command. With the barrier write command, we save a space in the command queue and the latency to dispatch the cache barrier command.

The storage controller fetches the IO command and extracts the <stream id, epoch id> pair, and barrier flag. The storage controller services the barrier write command as if the write command is immediately followed by the cache barrier command in the same stream. OPFTL persists the data blocks with respect to the storage order constraints within the stream and across streams.

4.3 OP-EXT4 Filesystem

We port EXT4 over OPIMQ to demonstrate how the filesystem can leverage OPIMQ. We call a newly ported EXT4 over OPIMQ as *OP-EXT4*. OPIMQ is filesystem agnostic. The filesystems other than EXT4, e.g., F2FS and XFS, can be ported over OPIMQ without significant effort.

For `fsync()`, OP-EXT4 writes the dirty pages, journal log blocks, and a journal commit block in the same order as EXT4 does. OP-EXT4 implements `fsync()` with three epochs: an epoch for writing dirty pages, an epoch for writing journal log blocks, and an epoch for writing the journal commit block. The application thread creates the first epoch, and the JBD thread creates the second and third epochs. All write requests associated with `fsync()` are ordered write. OP-EXT4 sets the last request in writing the dirty pages as a barrier write. This is to delimit an epoch. The same rule applies in writing the journal log blocks and writing the journal commit block.

OP-EXT4 begins servicing `fsync()` by writing the dirty pages associated with the journal transaction. After dispatching the write requests for dirty pages, the application thread wakes up the JBD thread without waiting for the completion of the write requests that have just been dispatched. In stock EXT4, the application thread blocks and waits for the completion of these write requests till it wakes up the JBD thread. The JBD thread in OP-EXT4 commits the journal transaction as follows: it first issues the ordered write for journal log blocks. After issuing the write requests for the journal log blocks, the JBD thread issues the write request for the journal commit block without waiting for the completion of the write requests for journal log blocks. In stock EXT4, the JBD thread waits for the completion of write requests for journal log blocks till it dispatches the write request for the journal commit block. After dispatching the write request for the journal commit block, the JBD thread issues `flush` command to ensure the durability of the `fsync()`. The `flush` command belongs to the JBD stream and forms an epoch by itself.

To ensure the durability of `fsync()`, it is critical that the

`flush` command is serviced after the write requests for journal log blocks and for the journal commit block are serviced. OP-EXT4 makes the flush command a separate epoch following the epoch of writing the journal commit block. When the storage device returns the `flush` command, the OP-EXT4 returns the `fsync()`.

Filesystem Interface. OP-EXT4 provides the file system interfaces `fbarrier()` and `fdatabarrier()` [34]. `fbarrier()` and `fdatabarrier()` are order-preserving counterparts of `fsync()` and `fdatasync()`. `fsync()` and `fdatasync()` guarantee the durability of the associated filesystem blocks when they return. `fbarrier()` and `fdatabarrier()` return after they dispatch all the associated commands to the storage device without waiting for them to become durable. `fbarrier()` and `fdatabarrier()` does not guarantee the durability. In particular, `fdatabarrier()` can be thought as a filesystem version of the memory barrier instruction, such as `mfence` [13] and `dmb` [9]. By interleaving `write()` calls with `fdatabarrier()`, the application ensures that the data blocks from `write()` requests that precede `fdatabarrier()` are made durable before those from `write()` requests that follow `fdatabarrier()`. `fdatabarrier()` is an effective method for controlling the storage order among the write requests without relying on heavily expensive `flush` or `fdatasync()`.

4.4 Block Device Layer

The block layer is responsible for assigning a stream id and an epoch id for each write request. We define a stream id and an epoch id at `struct bio`. When the block layer receives a write request from the filesystem, the block layer constructs a write request (`struct bio`) and assigns a stream id and an epoch id. For an ordered write, the block layer assigns the process id as its stream id. For an unordered write, the block layer assigns 0 as its stream id. Each stream maintains an epoch counter. An epoch counter is initialized to 0 when a stream is created. The block layer assigns the epoch counter value as the epoch id for each ordered write request. When a barrier write arrives, the block device layer assigns the current epoch counter to the barrier write as epoch id and increments the epoch counter by one. We define an epoch counter in the `task_struct`.

4.5 Guaranteeing Intra-Stream Storage Order

In OPIMQ, the host is responsible for clustering the write requests in an epoch at the same request queue, and the storage device is responsible for ensuring the storage order between the epochs, which are placed at different request queues.

Epoch split The work-stealing mechanism can make the write requests in an epoch placed over multiple request queues. We call this *epoch split*. When an epoch is split over multiple request queues, the command that delimits the epoch may arrive at the storage before some of the write requests in the same epoch are still in transit. Subsequently, the storage controller may delimit the epoch prematurely. If the storage controller delimits the epoch prematurely, the following epoch can be made durable ahead of the blocks in the preceding epoch that was prematurely delimited. As a result, the epoch split can compromise the storage order.

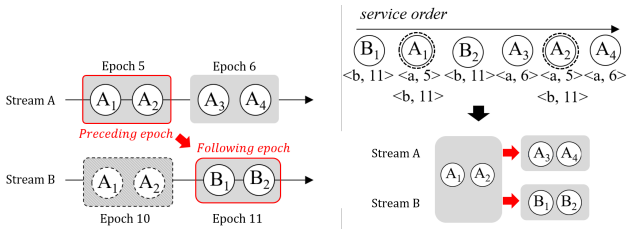
Epoch Pinning To avoid the epoch split, we propose a simple yet effective mechanism, *Epoch Pinning*. Epoch Pinning guarantees that the write requests at the same epoch are placed at the same request queue. Epoch Pinning decouples the request queue from the CPU and instead binds the request queue to the thread. Though the work-stealing mechanism migrates a thread from one core to another, the ordered write requests are placed at the same request queue where the first write request of the epoch is placed.

Implementation. A thread maintains a request queue id, called a *current request queue* (CRQ). In OPIMQ, a thread places the ordered writes at the current request queue. When a thread is migrated from one core to another, the current request queue is updated to the request queue that is bound to the new CPU that the thread is migrated to. However, the current request queue is updated only when there is no active epoch. If the thread has an active epoch, OPIMQ updates the current request queue when the active epoch is delimited. With Epoch Pinning, a request queue can be accessed by more than one CPU core. The request queue and command queue are protected by the spinlock, so that Epoch Pinning does not cause race conditions. According to our experiment, nor does it cause significant lock contention on the request queue (section 7.3).

Discussion Epoch Pinning allows the work-stealing module to freely migrate a thread from one core to another and yet allows the write requests in the same epoch clustered at the same request queue. The ccNVMe [25] also clusters the commands of a single transaction (similar to an epoch) at the same command queue. In ccNVMe, however, OS cannot migrate a thread if it has an open epoch. The easiest way to prevent epoch split is to statically bind a thread to a request queue. This approach may cause a severe load imbalance among the request queues. It can leave the request queue under lock contention since the request queue can be shared by multiple threads from different CPU cores.

4.6 Guaranteeing Inter-Stream Storage Order

Two epochs that need to be made durable in order can belong to the different streams and to different queues. It is called an inter-stream storage order. In OPIMQ, the host specifies the inter-stream storage order using *dual stream* write, and the storage device makes the data blocks durable, satisfying the ordering constraints in the dual stream write.



(a) dual-stream write (b) OPFTL

Figure 8: Dual Stream Write for handling the inter-stream storage order.

4.6.1 Dual-stream write

To ensure the inter-stream storage order among the write requests, we develop a special type of write request, *dual stream write*. Dual stream write is an ordered write with two pairs of <stream id, epoch id>. Having two pairs of id's, the dual stream write belongs to the two streams simultaneously. Subsequently, dual stream write should satisfy the storage order in both streams.

For convenience, we call the stream where the preceding epoch belongs as the preceding stream and the stream where the following epoch belongs as the following stream. When the filesystem creates the write requests in the preceding epoch, OPIMQ augments the newly created write requests with the secondary <stream id, epoch id> pair. The secondary stream id is the stream id of the following stream. The secondary epoch id is obtained from the current epoch counter of the following stream. When the preceding epoch is delimited, OPIMQ increases the epoch counters of the preceding stream and the following stream by one. OPIMQ block layer inserts the dual stream write at the request queue of the preceding stream.

Figure 8a illustrates the concept of the dual-stream write. The preceding stream A contains two epochs: $\{A_1, A_2\}$ and $\{A_3, A_4\}$. The following stream B contains one epoch: $\{B_1, B_2\}$. Streams A and B have stream ids a and b, respectively. There exist two storage order constraints: $\{A_1, A_2\} \mapsto \{A_3, A_4\}$ and $\{A_1, A_2\} \mapsto \{B_1, B_2\}$. For inter-stream storage order, OPIMQ creates the write requests in $\{A_1, A_2\}$ as the dual-stream writes. $\{A_1, A_2\}$ is assigned the epoch counter 5 as a preceding stream and epoch counter 10 for a following stream. The dual-stream writes A_1 and A_2 are assigned as <a, 5> and <b, 10>, respectively. The following epochs $\{A_3, A_4\}$ and $\{B_1, B_2\}$ are assigned by 6 and 11, respectively.

Figure 8b demonstrates how OPFTL can ensure the inter-stream order dependency between stream A and stream B using dual stream write. It assumes that requests from Figure 8a are placed across multiple command queues. OPFTL extracts the stream id and epoch id. OPFTL persists the data blocks with respect to the epoch id within a stream. In stream B, OPFTL persists B_1 and B_2 only after the A_1 and A_2 are persisted. In stream A, it persists A_3 and A_4 only after A_1 and A_2 are persisted. With the collaboration between dual stream write and OPFTL, the inter-stream storage order, $\{A_3, A_4\}$ and $\{B_1, B_2\}$, is successfully satisfied.

4.6.2 Many-to-one inter-stream order dependency

We can represent the many-to-one inter-stream order dependency using the dual stream write. This is because many-to-one inter-stream order dependency can be expressed as a conjunction of multiple one-to-one dependencies. Let us provide an example. Assume that three threads, T_1, T_2 and T_3, call `fsync()` on their own files simultaneously. The updated metadata associated with these `fsync()` calls is inserted at the same running transaction. Let the dirty pages updated by thread T_1, T_2, T_3 be D_1, D_2, and D_3, respectively. The journal commit block should become durable only after D_1, D_2, and D_3 become durable. There exists a 3:1 inter-stream storage order, $\{D_1, D_2, D_3\} \mapsto \{JC\}$. OPIMQ decomposes 3-to-1 order dependency into the three 1-to-1 order dependencies between a `fsync()` caller thread and JBD, as $\{D_1\} \mapsto \{JC\}$, $\{D_2\} \mapsto \{JC\}$, and $\{D_3\} \mapsto \{JC\}$. OPIMQ creates the write requests for D_1, D_2, D_3 as dual-stream writes. The secondary stream id of each of these dual stream writes is that of JBD.

4.6.3 Implementation

In the limited scenario, which we are interested in, e.g., filesystem journaling, there exists only one thread in the system whose stream can be a following stream in any inter-stream storage order dependency. The examples include JBD thread in EXT4 and checkpoint thread in XFS. We assume that the process id associated with this following stream does not change, and the filesystem can obtain the process id (stream id, equivalently) associated with the following stream when it constructs the dual stream write. In OP-EXT4, `fsync()` writes the dirty pages, journal log blocks, and a journal commit block in order. OP-EXT4 writes the dirty pages as a dual stream write. OP-EXT4 sets the secondary stream id of the dual stream write as the process id of the JBD thread. The secondary epoch id is obtained from the current epoch counter of the stream associated with JBD.

To represent a dual-stream write, OP-EXT4 adds a reference in the `bio` structure of the write request, which references the `task_struct` data structure of the following stream. For single-stream writes, this pointer is set to NULL. For dual-stream write, the filesystem sets this pointer to refer to the `task_struct` of the secondary stream. The block layer examines this pointer and assigns <stream id and epoch id> for the secondary stream.

5 Order Preserving FTL

5.1 Design Overview

We develop an FTL that ensures that the data blocks are persisted while satisfying the intra-stream and inter-stream

storage orders. We call it an Order-Preserving FTL, OPFTL. OPFTL flushes the writeback cache entries as the legacy storage controller does in any order, fully exploiting its internal parallelism. However, OPFTL controls the order in which the mapping table entries are updated so that the data blocks become durable in order from the host's point of view.

In OPFTL, an epoch can be in one of the four states: *active*, *closed*, *durable*, and *mapped* (Figure 9). An epoch is created and becomes in the *active* state when the order write arrives and the associated epoch does not exist. The epoch becomes *closed* if it is delimited. The epoch becomes *durable* if all of the associated data blocks become durable. The epoch goes into *mapped* state if FTL finishes updating the mapping table entries of all data blocks in the epoch.

OPFTL updates the mapping table differently for ordered write and for the unordered write when the data blocks in the writeback are written to the new physical location. For an unordered write, OPFTL updates the associated mapping table entry immediately. For an ordered write, OPFTL updates the associated mapping table entry only when the mapping table entries associated with the preceding epochs are all updated. For a dual stream write, OPFTL updates the associated mapping table entry only when the mapping table entries with the preceding epochs in both streams are all updated.

5.2 Order Preserving Mapping Table Update

OPFTL maintains a stream object for each stream. When the stream id of the incoming write command does not match any of the existing stream objects, the OPFTL creates a new stream object. A stream object consists of (i) a set of epochs, (ii) the most recently persisted epoch (RPE), and (iii) the set of delayed mappings. The stream object maintains a set of epochs. An entry in the epoch set is represented by the epoch id and the epoch state. The most recently persisted epoch denotes the most recent epoch whose data blocks are all made durable and whose mapping table entries are all updated. When an epoch goes into the mapped state, OPFTL updates the recently persisted epoch (RPE) of the associated stream to the newly mapped epoch. OPFTL deletes the mapped epoch from the epoch set in the background. The mapping information that cannot be reflected at the mapping table is temporarily stored at the *delayed mapping list*. Delayed mapping is mapping information for the data block that is made durable but whose mapping table entry is yet to be updated. Delayed Mapping is represented by <LPN, PPN, epoch id, sibling>. The sibling field is used only in the dual stream write. The sibling refers to the delayed mapping associated with the other stream.

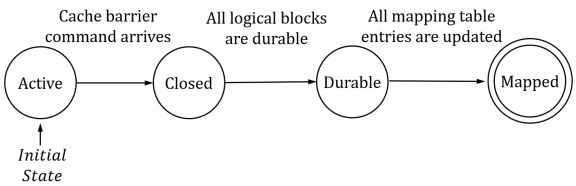


Figure 9: State Transition Diagram of an Epoch

OPFTL maintains two pairs of <stream id, epoch id> and a flag, *mappable*, for each block in the writeback cache. The second pair of <stream id, epoch id> is only for the dual stream write. The "mappable" flag denotes whether the mapping table entry can be updated or not. Each time when the storage controller transfers the data block to the writeback cache, it sets the mappable flag for the associated data block as follows: if the data block is associated with unordered write, mappable is set to TRUE. If the data block is associated with the ordered write, the storage controller examines if its preceding epoch is in the mapped state or not. If the preceding epoch is in the mapped state, the "mappable" flag is set to TRUE. Otherwise, it is set to FALSE.

OPFTL guarantees that the mapping table entries are updated satisfying the storage order constraint in a stream. A mapping table entry in the following epoch can be updated only after the preceding epoch in the same stream reaches the mapped state. An epoch reaches the mapped state only after all its data blocks are flushed and the mapping table entries are updated accordingly. A mapping table entry in the following epoch can be updated only after all mapping table entries in the preceding epochs are updated. Hence, the OPFTL preserves the intra-stream storage order.

When the storage controller flush the data block, it examines the "mappable" flag of the logical block. If "mappable" is TRUE, the storage controller updates the mapping table entry immediately. If "mappable" is FALSE, the storage controller inserts the entry <LPN, PPN, epoch id, sibling> to the delayed mapping list of the associated stream.

5.3 Sibling-aware Delayed Mapping

OPFTL inserts an entry to the delayed mapping list when it flushes the writeback cache entry, but it cannot update the mapping table immediately. For the dual stream write, OPFTL inserts *two* delayed mapping entries, <LPN, PPN, epoch id, sibling>, at the lists: one for the preceding stream and the other for the following stream, respectively. These two entries contain the same <LPN, PPN>. The "sibling" field of each entry refers to the other entry in the pair. For ordered write for the single stream, the sibling field is NULL.

OPFTL periodically scans the delayed mappings in each stream and examines if the preceding epoch of the entry is in *mapped* state. If the preceding epoch of the entry is in *mapped* state and if the mapping is associated with a single stream, i.e., the sibling field is NULL, OPFTL updates the mapping table with the delayed mapping entry and deletes it from the list. If the preceding epoch of the entry is in *mapped* state, but if the mapping is associated with the dual stream write, i.e., the sibling field is not NULL, OPFTL sets the sibling

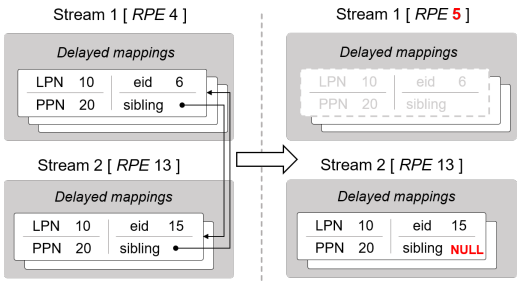


Figure 10: Sibling-aware delayed mapping

field of the other delayed mapping entry in the pair to NULL and deletes it from the list. In this manner, the mapping table entry is updated only when both epochs associated with the preceding stream and the following stream become mapped. This is called *sibling-aware delayed mapping*.

Fig. 10 shows the example. FTL inserts the two delayed mapping entries to streams 1 and 2. They are associated with the dual stream write <1,6>, <2, 15>. The storage controller inserts the entry, <LPN 10, PPN 20, eid 6, sibling> to the delayed mapping list of stream 1. It also inserts the entry, <LPN 10, PPN 20, eid 15, sibling> to the stream 2's delayed mapping list. These two entries refer to each other. The RPE values of each stream are 4 and 13, respectively. Assume that in stream 1, epoch 5 becomes "mapped". The RPE of stream 1 becomes 5. When OPFTL scans the delayed mappings in stream 1, it finds that the preceding epoch of the delayed mapping <LPN 10, PPN 20, eid 6, sibling> becomes mapped. This mapping is removed from the list. However, OPFTL does not update the mapping table. Instead, OPFTL sets the sibling field of the other mapping information associated with the dual stream write to NULL.

6 Crash Consistency

We test the crash consistency of OP-EXT4 with `CrashMonkey` [27]. We modified `CrashMonkey` to recognize the cache barrier to delimit an epoch when it encounters a cache barrier. We run two workloads: `rename_root_to_sub` and `create_delete`. For each workload, we generate 1,000 crash states and conduct crash consistency tests. OP-EXT4 passes all test cases for every crash state. OPIMQ guarantees that if a system crash occurs before an epoch is fully persisted in the storage, the subsequent epochs do not persist in the storage.

7 Evaluation

We compare OP-EXT4 (EXT4 port on OPIMQ) against vanilla EXT4, BarrierFS [34], and MQFS [25]. We implement OPIMQ in Linux 5.18.18. We use a 40 core server (DELL R740XD, 20 core/socket, Intel Xeon). We use a Samsung 980Pro NVMe SSD (2TB). Samsung 980Pro does not have order-preserving FTL. We add a 1.05% performance penalty to the Samusung 980Pro to simulate the order-preserving FTL. This penalty reflects the impact on throughput and is derived from measurements of an actual implementation of OPFTL on a Cosmos board (section 7.5). BarrierFS is configured to use a single request queue. We use the host's memory to simulate the PMR region in MQFS. We introduce the same latency as [25] in accessing the PMR region. MQFS journals only the file metadata, e.g., inode, and index blocks, but not the filesystem metadata, e.g., superblock or directory block. It is unclear if MQFS can correctly recover the filesystem in case of a system crash. We show the results of MQFS for the illustrative purpose.

7.1 Throughput

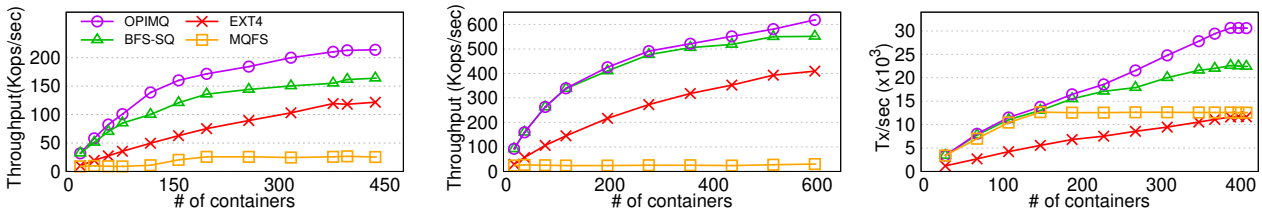
We examine OPIMQ's scalability against EXT4, BarrierFS, and MQFS in cloud environment. For the same setup as a cloud server, we create multiple containers with Docker [1] and run identical benchmarks at each container. We use Filebench [32] `varmail`, `dbench` [19], and `sysbench` [21] OLTP-insert workloads. Each container runs the benchmark with a single thread. We measure the total throughput of all containers. We increase the number of containers till the performance becomes saturated. The number of containers becomes larger than the number of CPU cores. Figure 11 shows the results.

`varmail`(Figure 11a) With 40 containers, OPIMQ outperforms EXT4, BFS-SQ and MQFS by 1.1 ×, 3.0 ×, 5.2 ×, respectively. OPIMQ shows 1.3 ×, 1.8 × and 6.4 × performance against BFS-SQ, EXT4, and MQFS when the number of containers is 450. In BarrierFS, because it can use only one IO queue pair, it achieves 24 % lower throughput than the OPIMQ, which employs all 40 IO queue pairs.

`dbench`(Figure 11b) OPIMQ outperforms EXT4 and MQFS by 2.8×, 6.1 × when the number of containers is equal to the CPUs. When the number of concurrent containers is 600, OPIMQ shows 1.5× and 1.1× 20× performance against EXT4, BFS-SQ and MQFS, respectively.

`sysbench`(Figure 11c) OPIMQ shows 2.9× and 1.4× performance against EXT4 and BFS-SQ at 40 concurrent containers. When there are 400 containers, OPIMQ outperforms EXT4 by 2.6× and BFS-SQ by 1.4×. For MQFS, OPIMQ has a 0.2 % less performance when the number of containers is 40. However, OPIMQ has a saturated throughput that is around 1.8 times higher than MQFS.

Analysis. OPIMQ fully utilizes all CPUs and all request queues rendering the best performance in all three workloads. BarrierFS uses only one request queue. EXT4 uses transfer-and-flush to ensure the storage order. The OPIMQ renders the best performance in `varmail`. This is because OPIMQ significantly improves the `fsync()` performance via reducing the overhead associated with enforcing the storage order in the journal commit. `dbench` calls a fewer `fsync()` calls compared to the varmail. The number of `fsync()` calls ac-



(a) `Filebench varmail` (b) `Dbench` (c) `Sysbench OLTP-insert` on `MySQL`

Figure 11: Performance of `varmail`, `dbench`, and `Sysbench OLTP-insert` workloads with varying the number of docker containers. DELL PowerEdge R740XD, 20core/socket, 2 sockets, Intel Xeon Gold 6230 2.1G. Samsung 980Pro

counts for 5.2% of total file operations. The advantage of using a multi-queue is not significant in `dbench`. In `dbench`, BFS-SQ and OPIMQ renders similar performance. MQFS generates an excessive number of transaction commits. In contrast, EXT4, OP-EXT4, and BFS use compound transactions in journaling, which allow flush requests to be merged into a single one. MQFS calls 5.1 ×, 1.6 × and 1.2 × more flushes than OPIMQ in `varmail`, `dbench` and `sysbench`, respectively.

7.2 Latency: `fsync()`

We analyze the latency of `fsync()` under FIO [3]. We also examine the `fsync()` latency with three workloads: Filebench `varmail`, `dbench`, and `sysbench` OLTP-insert workload.

FIO benchmark We generate a 4 KByte random write followed by `fsync()`. Figure 12 shows the result. We divide the total latency of `fsync()` into four parts: T_{HOST}, T_{DMA}, T_{Flush}, and T_{FUA}. T_{HOST} is the amount of time that the host spends on organizing and dispatching IO requests as well as switching contexts. T_{DMA} is the amount of time the host waits for the DMA transfer of the write requests. T_{Flush} and T_{FUA} denote the amount of time the host has to wait for the storage device to finish processing the flush request and the write request with the `FUA` flag, respectively. Figure 12 shows the `fsync()` latency of OPIMQ and EXT4. The latency of `fsync()` is 2.2 milliseconds in OPIMQ and 6.7 msec in EXT4. The `fsync()` latency in OPIMQ is 1/3 of that of EXT4. The T_{DMA} of OPIMQ is about 1/2 of that of EXT4. This is because OPIMQ is free from wait-on-transfer for all write requests generated by `fsync()`. EXT4 requests a flush command to preserve the storage order between JL and JC. OPIMQ also requests a flush to ensure data durability. For both OPIMQ and EXT4, T_{Flush} is 1.8 msec. EXT4 sets the FUA flag to the journal commit block to ensure the durability of the data. T_{FUA} is 4.2 milliseconds, which accounts for 63 % of the total latency of `fsync()` in EXT4. OPIMQ, on the other hand, does not dispatch the FUA write request, so T_{FUA} is zero.

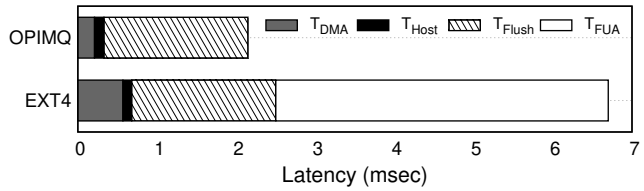
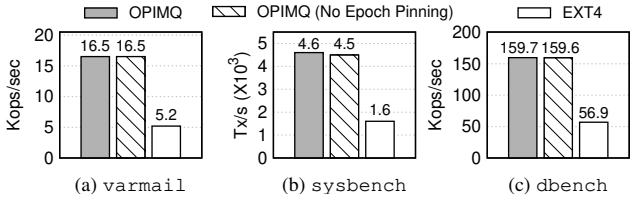


Figure 12: Dissection of `fsync()` latency, 4KByte random write followed by `fsync()`. Samsung 980Pro, linux 5.18.18

Tail latency Table 2 shows tail latency of `fsync()` `varmail`, `dbench` and `sysbench` when the number of containers is 40. OPIMQ reduces the average latency of `fsync()` by 67.7%, 69.5%, and 60.9% at `varmail`, `dbench`, and `sysbench` against to EXT4, respectively. OPIMQ shows shorter tail latencies than EXT4 at 99th, 99.9th, and 99.99th percentiles. In the case of the 99.99th percentile, the latency in EXT4 is 2.1 ×, 3.2 ×, and 1.5 × longer than in OPIMQ, at `varmail`, `dbench`, and `sysbench` respectively.



(a) `varmail` (b) `sysbench` (c) `dbench`

Figure 13: Epoch Pinning: Throughput. 40 concurrent docker containers. Samsung 980 Pro.

7.3 Epoch Pinning

Throughput We examine the benchmark throughput with and without Epoch Pinning. Without Epoch Pinning, write requests are inserted at the request queue associated with the current CPU. We use three benchmarks; `varmail`, `dbench`, and `sysbench` workloads. We deploy 40 client threads. Figure 13 illustrates the results. In all three benchmarks, Epoch Pinning does not cause any significant performance overhead.

	Mean		Median		99th		99.9th		99.99th	
	EXT	OPI	EXT	OPI	EXT	OPI	EXT	OPI	EXT	OPI
Varmail	12.7	4.1	12.6	3.9	16.7	8.1	32.3	14.3	346.6	162.2
Dbench	11.8	3.6	11.9	3.6	13.4	7.5	25.0	11.2	55.2	17.1
Sysbench	6.9	2.7	6.3	2.1	12.8	6.0	13.6	6.6	22.6	14.8

Table 2: Tail latency (msec) of `fsync()` in `varmail`, `dbench`,and `sysbench` (EXT: EXT4, OPI: OPIMQ)

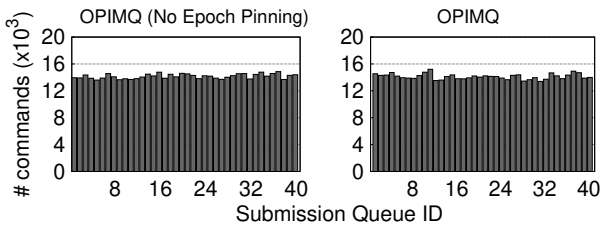


Figure 14: Epoch Pinning: Command Distribution over 40 Submission Queues. Samsung 980 Pro.

IO load balance We examine if Epoch Pinning introduces any significant load skew among the submission queues of the storage device. We run the `varmail` with forty threads and examine the number of commands fed to individual command queues. We use `ftrace` [31] to collect the number of commands placed at the individual command queues.

Figure 14 illustrates the result. The x-axis and y-axis represent the submission queue ID and the total number of IO commands submitted to the command queue, respectively. We can hardly observe the difference in the number of commands submitted to the SQs between the OPIMQ without Epoch Pinning and with Epoch Pinning.

Analysis We find that the epoch is split across the queue very rarely and subsequently Epoch Pinning seldom places the request to a different request queue from the one associated with the current CPU. In `varmail`, `dbench`, and `sysbench`, 99.99% of the epochs consist of only one request. In the three workloads, there occurs 6, 0 and 0 epoch split, respectively.

7.4 Work-Stealing and OPIMQ

We examine how OPIMQ and ccNVMe [25] distribute the load among the CPUs. We run 20 threads in the 40-core machine. Each thread calls 4KB `write()` for 100 times then calls `fsync()`. OPIMQ and ccNVMe render 112KIOPS and 57KIOPS, respectively.

Figure 15 shows the CPU utilizations of all CPUs. We sort the cores with respect to the CPU utilization in ascending order. The average CPU utilization for MQFS is 34% and for OPIMQ is 22%, respectively. OPIMQ balances the load over CPUs more evenly than ccNVMe does.

7.5 Overhead of Order Preserving FTL

We examine the performance and the address translation overhead of OPFTL and the page mapping FTL. We use the

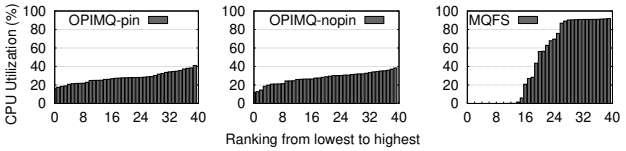


Figure 15: CPU load balancing: OPIMQ vs ccNVMe [25]. 20 threads in the 40-core machine.

	Page Mapping FTL	OPFTL
Throughput (Kops/sec)	1.91	1.89
Latency (usec)	2.65	2.65

Table 3: Overhead of OPFTL, OpenSSD, 4KByte random write followed by `fsync()` for 300 seconds, five streams.

Cosmos OpenSSD (230GByte, 8 channels, and 1 core) [15]. We generate the 4 KByte random `write()` followed by `fsync()` [3]. We measure the performance when the number of concurrent streams is five.

Table 3 illustrates the result. The throughput(ops/sec) of OPFTL is 1.05% lower than that of page mapping. OPFTL's address translation latency is similar to page mapping FTL's latency. The overhead of ensuring the storage order in OPFTL is negligible. OPFTL flushes the writeback cache in any order while updating the associated mapping table entries in the storage order. With this design feature, OPFTL can ensure the storage order among the epochs, while fully exploiting the internal hardware parallelism inside SSD.

Discussion We cannot conduct experiments with more than five streams because the host PC compatible with the Cosmos board has only 4 cores. However, we expect that even with an expansion of streams to 200, the overhead of OPFTL will not undergo significant changes compared to when there are only 5 streams. The FTL operates in a single thread since the OpenSSD board we use is equipped with a single core. Therefore, there are no lock contentions in OPFTL. Increasing the number of concurrent streams beyond five does not result in any additional lock contention. Memory pressure for OPFTL is insignificant. Each entry of the per-stream delayed mapping table is 32 Byte.

8 Conclusion

In this work, we propose the order-preserving IO stack for multi-queue block device, *OPIMQ*. OPIMQ exploits the notion of *stream* and *epoch*, so that the write order is preserved in a multi-queue environment. OPIMQ improves the overall performance of containers in Filebench `varmail`, `dbench`, and `sysbench` by 2.9 ×, 2.8 ×, and 2.9 × respectively against to the default Linux IO stack. This storage-centric approach saves the host from the overhead of waiting for the data blocks to be transferred to the storage and from the overhead of waiting for the flush command to be serviced.

9 Acknowledgements

We extend our gratitude to our shepherd, Matias Bjørling, for his guidance in refining the final version of this paper. We also appreciate the insightful feedback provided by the anonymous reviewers, which greatly enhanced the quality of this work. This work was in part supported by IITP, Korea (RS-2024-00459026) and NRF, Korea (NRF-2020R1A2C3008525).

References

[1] Docker. https://www.docker.com/.

[2] EXT4 mount options. https://lwn.net/Articles/283161/.

[3] FIO. https://fio.readthedocs.io/en/latest/fio_doc.html/.

[4] Intel Optane SSD 900P Series. https://www.intel.com/content/www/us/en/products/memory-storage/solid-state-drives/consumer-ssds/optane-ssd-9-series.html.

[5] Multi-Queue Block IO Queueing Mechanism (blk-mq). https://docs.kernel.org/block/blk-mq.html.

[6] NVM Express Specification. https://nvmexpress.org/specifications/.

[7] Serial ATA Revision 3.3 Specification. http://www.usedsite.co.kr/pds/file/SerialATA_Revision_3_0_RC11.pdf.

[8] SK hynix e-NAND Product Family eMMC5.1 Compatible. https://community.nxp.com/pwmxy87654/attachments/pwmxy87654/qoriq-grl/8936/1/SKhynix1znm_128Gb_eMMC51_ver1.0.pdf/.

[9] ARM. *Architecture Reference Manual, ARMv8, for ARMv8-A architecture profile*, 2018.

[10] Robert D Blumofe and Charles E Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM (JACM), 1999*, 46(5):720–748.

[11] Yun Sheng Chang and Ren Shuo Liu. OPTR:Order-Preserving Translation and Recovery Design for SSDs with a Standard Block Device Interface. In *Proc. of 2019 USENIX Annual Technical Conference (ATC), 2019*.

[12] Vijay Chidambaram, Thanumalayan Sankaranarayana Pillai, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Optimistic crash consistency. In *Proc. of the 24th ACM Symposium on Operating Systems Principles (SOSP), 2013*.

[13] Jaejin Lee Fang, Xing and Samuel P. Midkiff. Automatic fence insertion for shared memory multiprocessing. In *Proc. of the 17th annual international conference on Supercomputing, 2003*.

[14] CHEN Feng, LEE Rubao, and ZHANG X. Essential roles of exploiting internal parallelism of flash memory based solid state drives in high-speed data processing. 2011.

[15] Kibin Park Jinwoo Jeong Jaewook Kwak, Sangjin Lee and Yong Ho Song. Cosmos+ OpenSSD: Rapid Prototype for Flash Storage Systems. *ACM Transactions on Storage (TOS), 2020*, 16(3):1–35.

[16] JEDEC. Universal Flash Storage (UFS) 4.0, 2022.

[17] N. Jhala, J. Li, and Y. Zhou. Multi-channel and multi-way controller architecture for high-performance flash memory storage. In *Proc. of the International Symposium on Performance Analysis of Systems and Software (ISPASS), 2012*.

[18] Kaushal Yadav Joshi, Kanchan and Praval Choudhary. Enabling NVMeWRR support in Linux Block Layer." 9th USENIX Workshop on Hot Topics in Storage and File Systems. In *Proc. of the 9th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), 2017*.

[19] Karama Kanoun, Henrique Madeira, Mario Dalcin, Francisco Moreira, and Juan Carlos Ruiz Garcia. Dbench*(dependability benchmarking). In *Proc. of 5th European Dependable Computing Conference (EDCC), 2005*.

[20] Y. Kim, H. Hwang, and K. Yoo. A multi-channel and multi-way controller for high-performance nand flash-based storage systems. In *Proc. of the International Conference on Embedded Software and Systems (ICESS), 2013*.

[21] Alexey Kopytov. Sysbench: a system performance benchmark. *http://sysbench. sourceforge. net/*, 2004.

[22] Changman Lee, Dongho Sim, Jooyoung Hwang, and Sangyeun Cho. F2FS: A New File System for Flash Storage. In *Proc. of 13th USENIX Conference on File and Storage Technologies (FAST), 2015*.

[23] S. Lee and I. Park. Multi-channel and multi-way controllers for high-performance nand flash-based storage systems. *Journal of Systems Architecture, 2015*, 61(9):480–492.

[24] Xiaojian Liao, Youyou Lu, Erci Xu, and Jiwu Shu. Write dependency disentanglement with HORAE. In *Proc. of 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2020*.

[25] Xiaojian Liao, Youyou Lu, Zhe Yang, and Jiwu Shu. Crash Consistent Non-Volatile Memory Express. In *Proc. of 28th ACM Symposium on Operating Systems Principles (SOSP), 2021*.

[26] MK McKusick and GR Ganger. A Technique for Eliminating Most Synchronous Writes in the Fast Filesystem. In *Proc. of 1999 USENIX Annual Technical Conference (ATC), 1999*.

[27] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, and Vijay Chidambaram. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2018)*, Carlsbad, CA, 2018. USENIX Association.

[28] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Part V Transaction Management - Database Management Systems*, volume 3. McGraw-Hill New York, 2003.

[29] H Rev. SCSI Commands Reference Manual. http://www.seagate.com/files/staticfiles/support/docs/manual/Interface%20manuals/100293068h.pdf/, Jul 2014. Seagate.

[30] Mendel Rosenblum and John K Ousterhout. The design and implementation of a log-structured file system. *ACM Transactions on Computer Systems (TOCS), 1992*, 10(1):26–52.

[31] Steven Rostedt. Finding origins of latencies using ftrace. In *Proc. of Real Time Linux Workshop (RT Linux WS), 2009*.

[32] Vasily Tarasov, Erez Zadok, and Spencer Shepler. Filebench: A flexible framework for file system benchmarking. *USENIX login, 2016*, 41(1):6–12.

[33] Stephen C Tweedie et al. Journaling the Linux ext2fs filesystem. In *Proc. of the 4th Annual Linux Expo, 1998*.

[34] Youjip Won, Jaemin Jung, Gyeongyeol Choi, Joontaek Oh, and Seongbae Son. Barrier-Enabled IO Stack for Flash Storage. In *Proc. of 16th USENIX Conference on File and Storage Technologies (FAST), 2018*.

[35] Jiwon Woo, Minwoo Ahn, Gyusun Lee, and Jinkyu Jeong. D2FQ:Device-Direct Fair Queueing for NVMe SSDs. In *Proc. of 19th USENIX Conference on File and Storage Technologies (FAST), 2021*.

[36] Zhe Yang Xiaojian Liao and Jiwu Shu. Rio: Reliable i/o isolation for containerized cloud services. In *Proc. of the 2023 European Conference on Computer Systems (EuroSys), 2023*.

A Artifact Appendix

Abstract

The artifact package for "OPIMQ: Order-Preserving IO Stack for Multi-Queue Block Device" includes the source code, benchmarks, and scripts necessary to evaluate the OPIMQ framework. The key idea is to optimize `fsync()` operations by preserving I/O order using stream IDs, epoch IDs, and barrier flags. This package enables users to reproduce the experiments presented in the paper and validate the proposed techniques.

Scope

This artifact allows validation of claims related to the order-preserving I/O stack's effectiveness in reducing `fsync()` latency and maintaining write-order dependencies in multi-queue environments. It can be used to compare OPIMQ with baseline systems such as EXT4 and other order-preserving solutions like MQFS.

Contents

The artifact includes:

- Source code for OPIMQ and OPFTL (on GitHub: https://github.com/ESOS-Lab/OPIMQ.git).
- Precompiled kernels for different configurations (e.g., OPIMQ enabled, baseline).
- Experiment scripts for benchmarks (e.g., `fsync()` latency, epoch pinning, and container throughput).
- Instructions for quick validation and detailed experimental setups.

Hosting

The artifact is hosted on GitHub under the repository https://github.com/ESOS-Lab/OPIMQ.git.

- **Branch**: main
- **Commit Version**: Ensure you use the latest commit for the experiments.

Requirements

- **Hardware**: Tested on Samsung 980 Pro NVMe SSD (1TB).
- **Software**: Linux kernel 5.18.18, Docker, tmux, FIO benchmark tool.
- **Access Credentials**: SSH client and access to the reproducibility server.

No special hardware beyond a standard NVMe SSD is required for reproducing most experiments.